

High Performance Computing

A.A. 2025/2026

Cozzini Stefano
Tornatore Luca

Valle Alessio - SM3600018

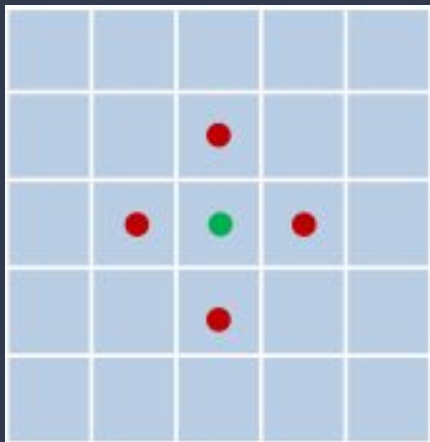
CdS Scientific and Data Intensive Computing

Università degli studi di Trieste

The problem



5 points stencil

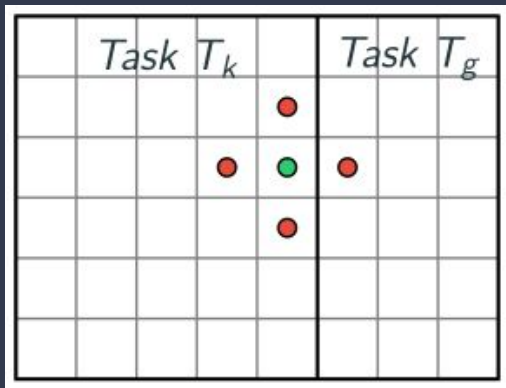


The objective of this project is to implement a parallel 5-point stencil computation utilizing OpenMP and MPI.

Stencil computations have diverse applications including the numerical solution of partial differential equations, such as the heat equation.

Each grid point $x_{ij}(t)$ is updated iteratively based on its own value at the previous time step, $x_{ij}(t - 1)$, and the values of its neighbors.

Border Regions



To enable parallel execution, the computational grid can be partitioned into subdomains, with each subdomain assigned to a separate process.

Since processes must exchange border data at each iteration, we need the processes to communicate with each other.

The idea is to enlarge the local patches to include the halo regions (the border of the local patch).

Problems to solve

- **Efficient communication pattern:**
 - Non-Blocking Communication
- **Efficient memory management:**
 - Avoid unnecessary allocations and copies
 - Cache locality among processes
- **Computation / Communication Overlap:**
 - Border computation parallel to inner computation
 - Using irecv and isend for communication

First implementation



The data

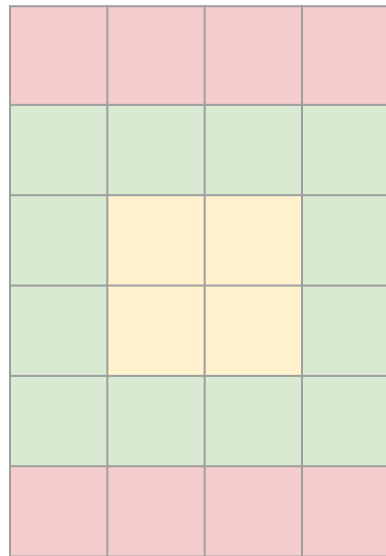
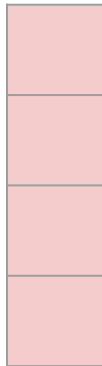
```
typedef struct
{
    double *restrict data;
    vec2_t size;
} plane_t;
```

```
buffers_ptr[t][NORTH] = &(planes_ptr[t].data[0]);
buffers_ptr[t][SOUTH] = &(planes_ptr[t].data[(y_size+1)*x_size]);
```

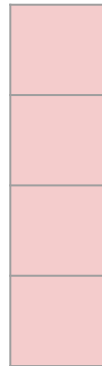
```
borders_ptr[t][NORTH] = &(planes_ptr[t].data[1 * x_size + 0]);
borders_ptr[t][SOUTH] = &(planes_ptr[t].data[y_size * x_size]);
```

`plane_t planes[2]`

`buffers[2]`



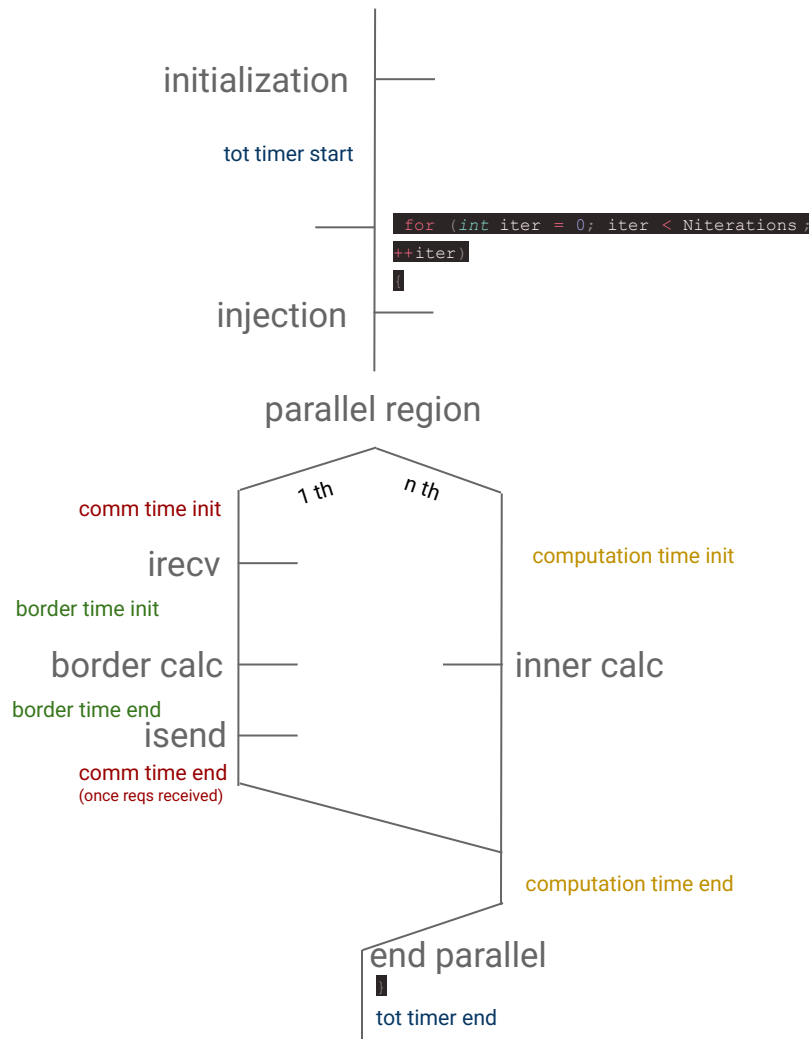
`buffers[2]`



```
planes_ptr[OLD].data = (double *)malloc(frame_size * sizeof(double));
if (planes_ptr[OLD].data == NULL)
    return 1;
else
    memset(planes_ptr[OLD].data, 0, frame_size * sizeof(double));
```

The loop

```
#pragma omp for collapse(2) schedule(static) nowait
for (uint j = 2; j < ysize-1; j++)
    for (uint i = 1; i < xsize; i++)
    {
```



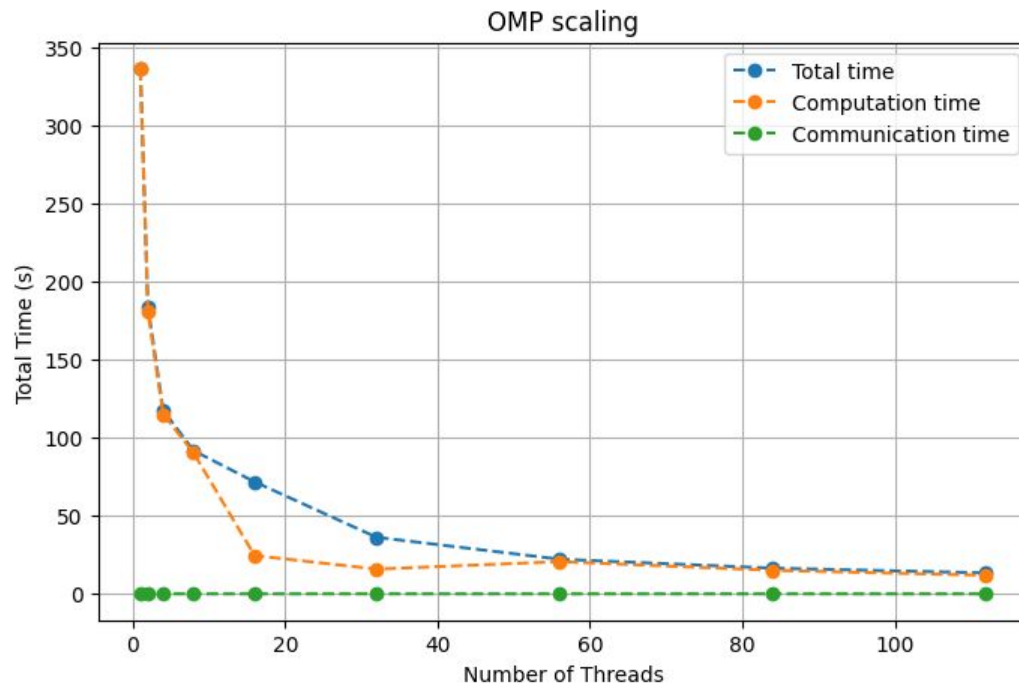
Comm thread

```
#pragma omp single nowait
{
    for (int i = 0; i < 4; ++i) {
        int x or y = i >> 1;
        MPI_Irecv(buffers[!current][i], decomposedS[x_or_y], MPI_DOUBLE, neighbours[i], iter, myCOMM_WORLD, &reqs[4+i]);
    }
    t_start_calc_iter = MPI_Wtime();
    for (int i = 0; i < 4; ++i) {
        int x or y = i >> 1;
        double const *old_border = border_ptr[current][i];
        double const *old_buffer = buffers[current][i];
        double *new_border = border_ptr[!current][i];
        double *momentary_buffer = NULL;
        if (x or y) {
            momentary_buffer = (double *)malloc(decomposedS[_y_] * sizeof(double));
            send_buffers[i] = momentary_buffer;
        } else {
            send_buffers[i] = new_border;
        }
        update_border_calc(i, decomposedS, old_border, old_buffer, new_border, momentary_buffer);
        atomic_store_explicit(&ready[i], 1, memory_order_release);
        MPI_Isend(send_buffers[i], decomposedS[x_or_y], MPI_DOUBLE, neighbours[i], iter, myCOMM_WORLD, &reqs[i]);
    }
}
```

Thread scaling

thread:
1, 2, 4, 8, 16, 32, 56, 84, 112

NODES=1
N_TASKS_PER_NODE=1
TOTAL_TASKS=1
N_STEPS=1000
GRID_SIZE_X=15000
GRID_SIZE_Y=15000



Strong scaling

grid 15000*15000
OMP_THREADS=14
CORES_PER_NODE=112
NTASKS_PER_NODE=8

1 NODES:

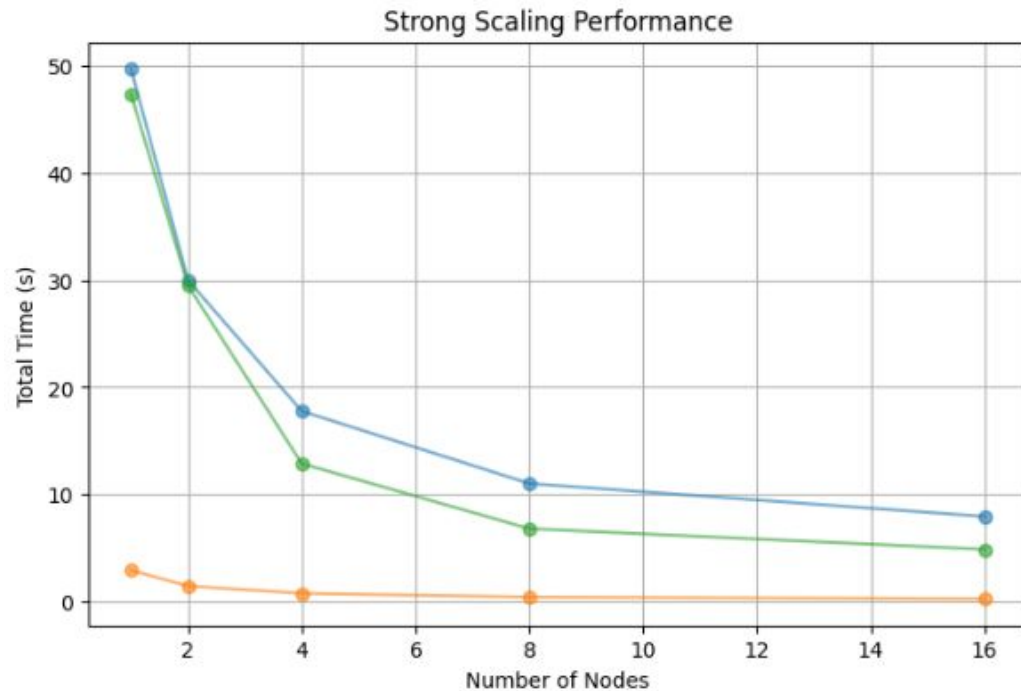
total time, Computation
49.649266, 47.317554

2 NODES:

total time, Computation
29.968883, 29.482886

4 NODES:

total time, Computation
17.767935, 12.875528



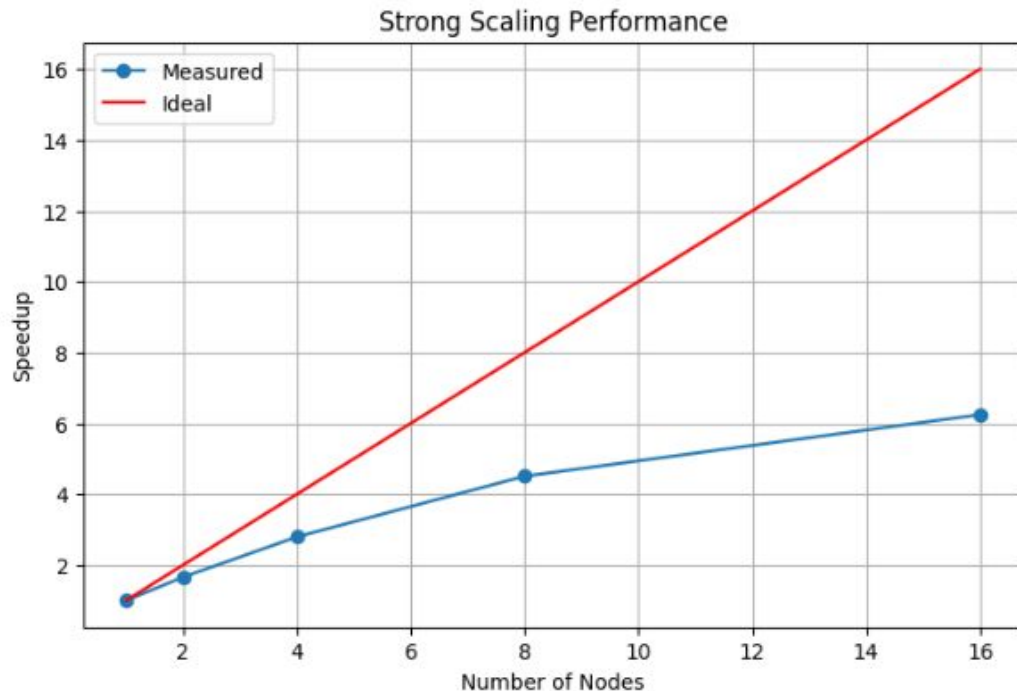
Strong scaling

```
grid 15000*15000  
OMP_THREADS=14  
CORES_PER_NODE=112  
NTASKS_PER_NODE=8
```

```
1 NODES:  
total time, Computation  
49.649266, 47.317554
```

```
2 NODES:  
total time, Computation  
29.968883, 29.482886
```

```
4 NODES:  
total time, Computation  
17.767935, 12.875528
```



Weak scaling

N_STEPS=500
TASKS_PER_NODE=8
OMP_THREADS=14
LOCAL_SIZE=4000

for NODES in 1 2 4 8 16

1N 8t:

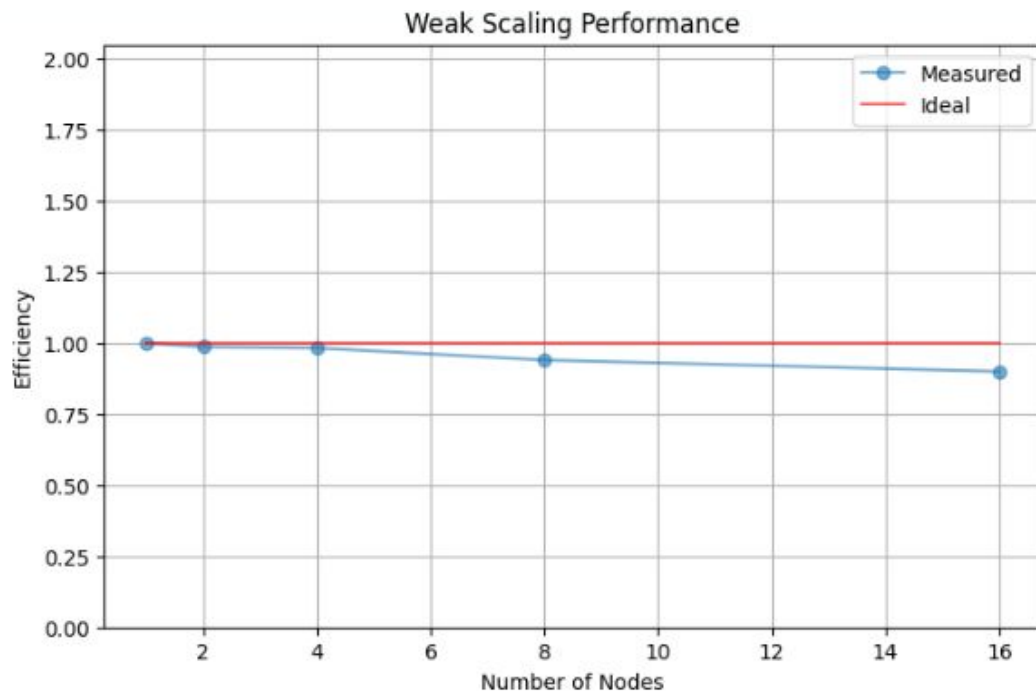
```
total time, Computation  
63.476983, 29.143855
```

2n 16t:

```
total time, Computation  
58.472084, 26.398154
```

4n 32t:

```
total time, Computation  
65.025389, 31.610051
```



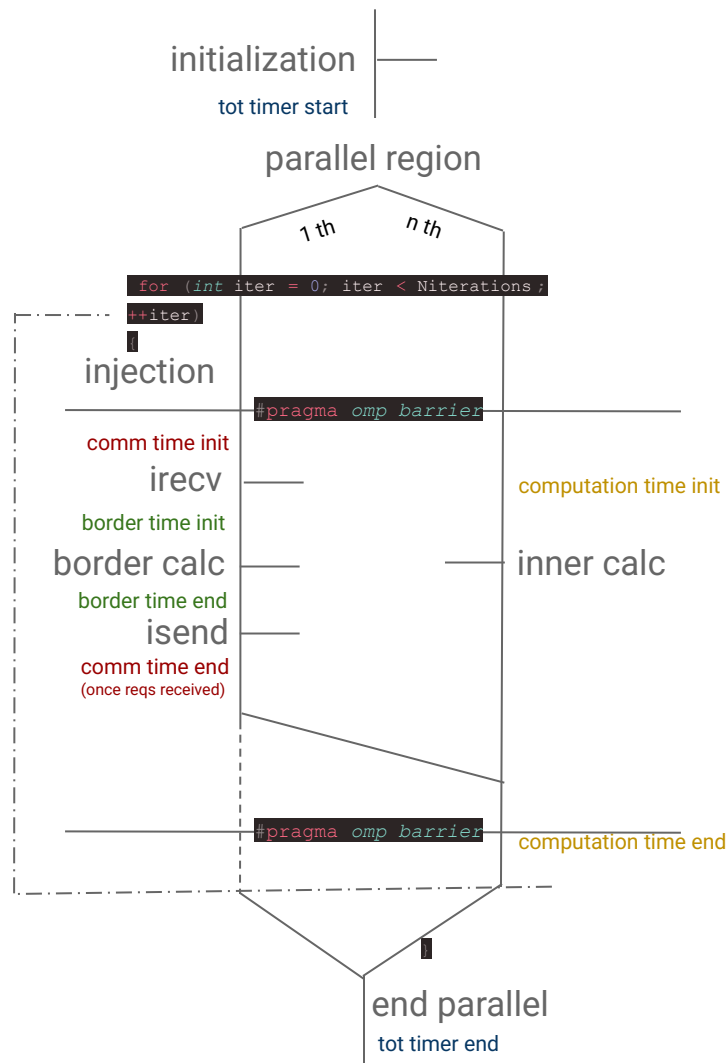
Current implementation

A dark blue diagonal gradient bar that starts from the bottom-left corner and extends towards the top-right corner, covering the lower half of the slide.

The loop

ALSO CHANGED:

- all internal allocation outside the parallel region;
- no malloc and no free in the cycle.



Thread Scaling

grid 16384*16384

thread:

1, 2, 4, 8, 16, 32, 56, 84, 112

NODES=1

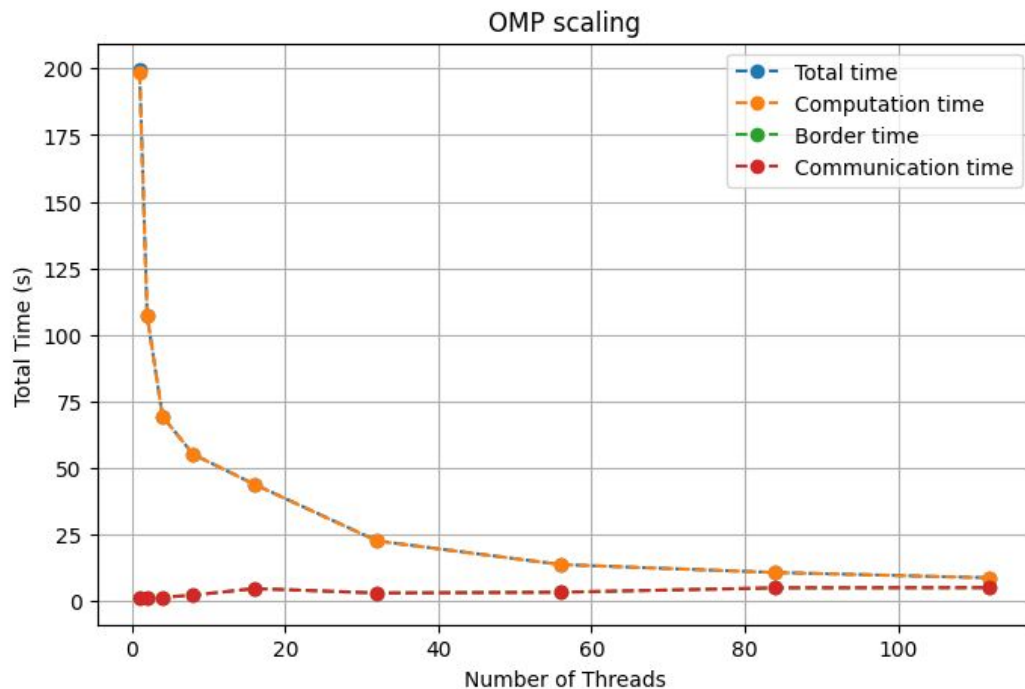
N_TASKS_PER_NODE=1

TOTAL_TASKS=1

N_STEPS=500

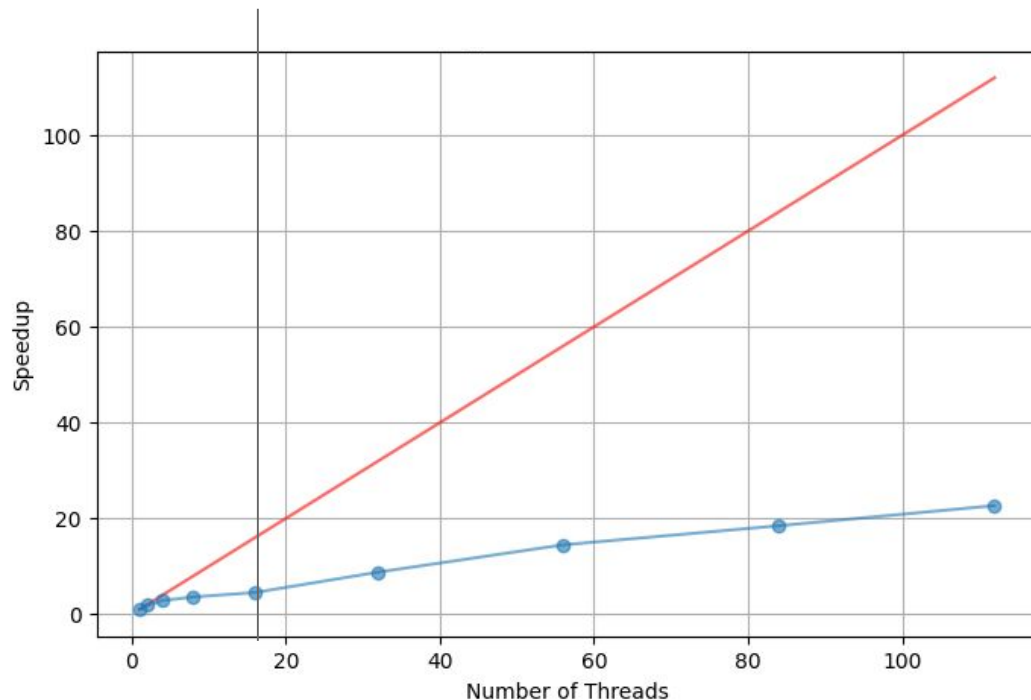
GRID_SIZE_X=16384

GRID_SIZE_Y=16384



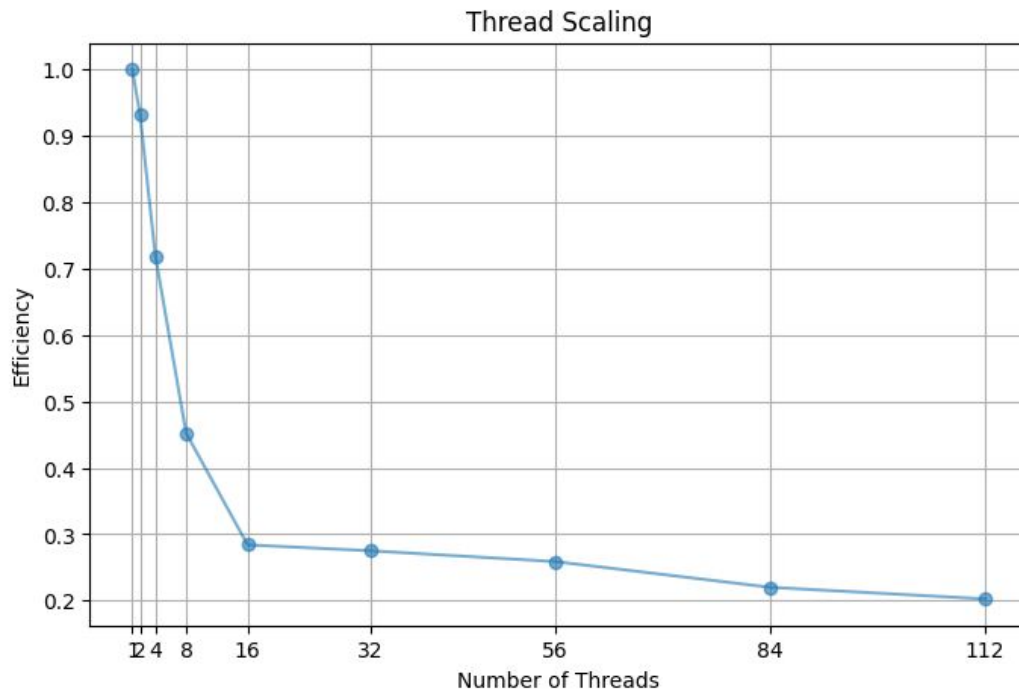
Thread scaling

- Saturation of busses to ram;
- need to go through L3 cache;
- small increase after 14 due to more numa regions being used;
- set up as closed the disposition of the tasks.



Thread scaling

- No flattening of the efficiency, probably due to the parallel nature of the calculations of border and inner.
- Calculation of the border doesn't get split in more parts.



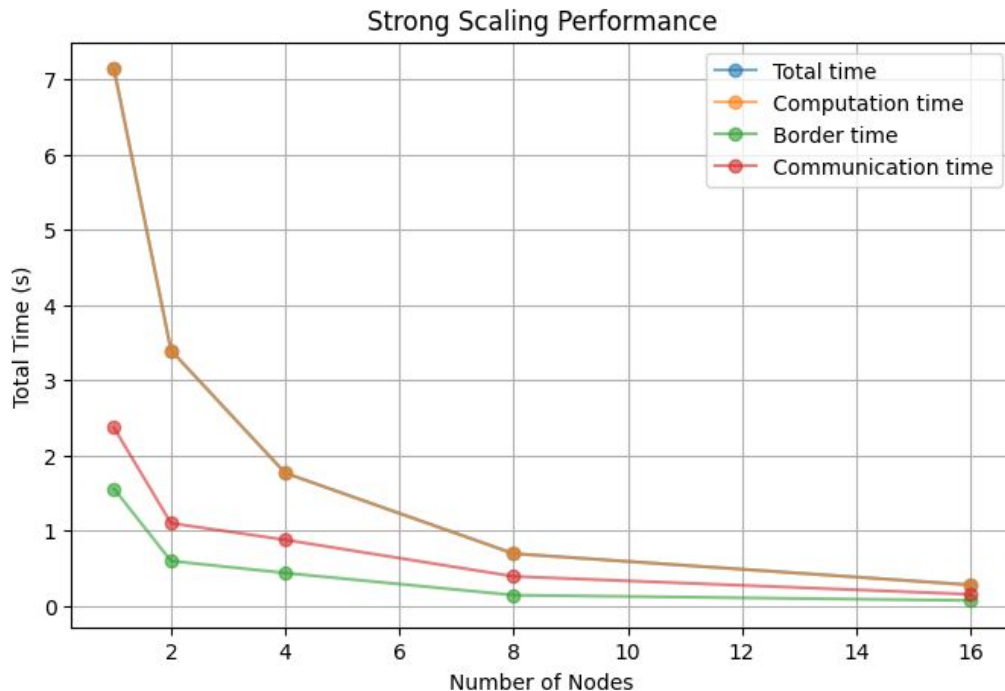
Strong scaling

N_STEPS=500
GRID_SIZE_X=16384
GRID_SIZE_Y=16384
OMP_THREADS=14
CORES_PER_NODE=112
NTASKS_PER_NODE=8

```
1 NODES:  
tot time,comput, border, comm  
7.157581,7.153436,1.574018,2.418358
```

```
2 NODES:  
3.413603,3.410817,0.577413,1.004723
```

```
16 NODES:  
0.297721,0.295379,0.072133,0.166406
```



Strong scaling

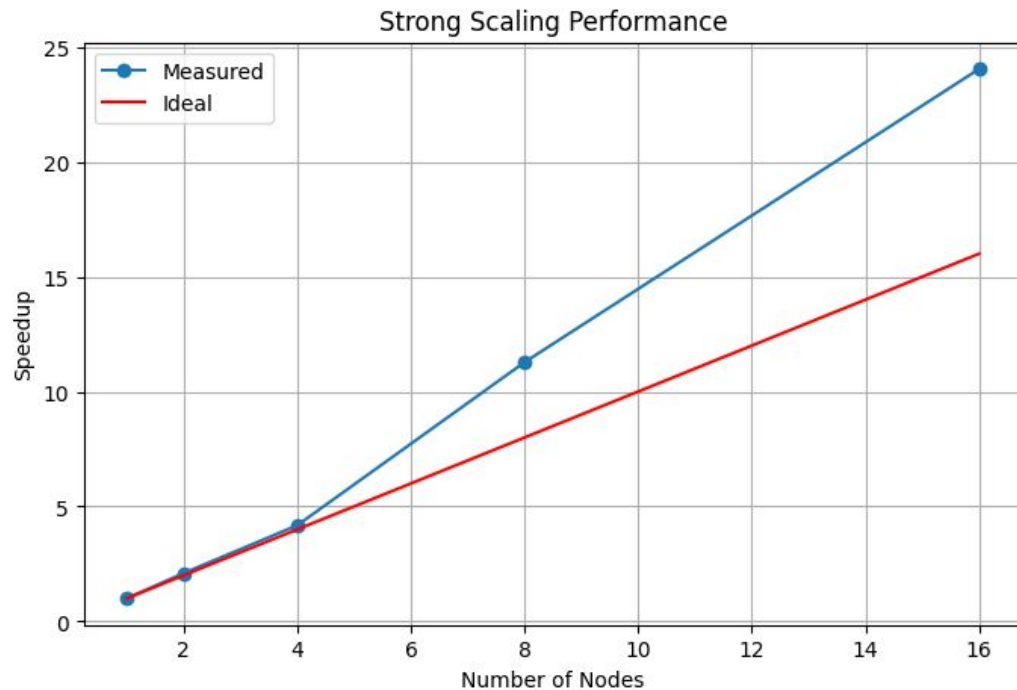
1 NODE:
94,292 % of all cache refs

2 NODES:
89,352 %

4 NODES:
79,325 %

8 NODES:
43,668 %

16 NODES:
4,519 %



Strong scaling

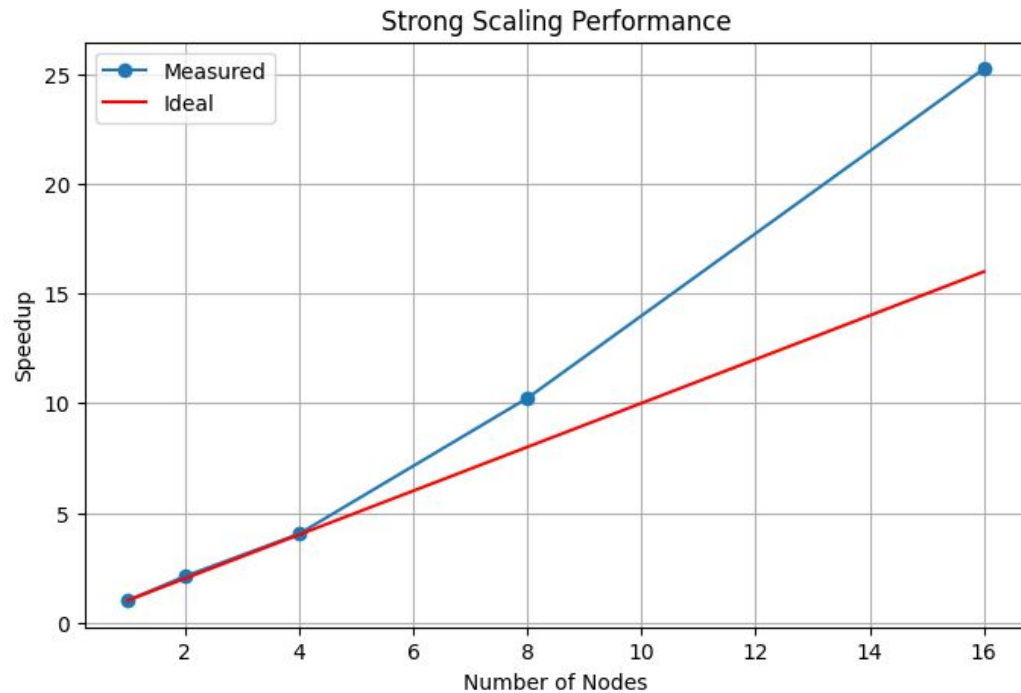
1 NODE:
94,144 % of all cache refs

2 NODES:
89,433 %

4 NODES:
79,258 %

8 NODES:
43,275 %

16 NODES:
6,233 %



Space Occupation

Matrix= $16.384 \times 16.384 = 268.435.456$ squares
total size= Matrix*`sizeof(Double)`=Matrix*8= 2.147.483.648 Bytes

$2.147.483.648 / 16 = 134.217.728$ bytes per node.
 $134.217.728 / 112 = 1.198.372$ bytes per core.

At 16 nodes a matrix can live in each l2 cache.

Cycle Losses

(keeping calculations rough):

4 misses per stencil point: 3 read and 1 write

Total cache misses=268M*4=1.073G misses

1 NODE: 94,292% misses

Ram: $1.073G \times 0,943 \times 350 = 350G$ cycles

L3: $1.073G \times 0,057 \times 100 = 61G$ cycles

16 NODE: 4,519% misses

Ram: $1.073G \times 0,045 \times 350 = 16,9G$ cycles

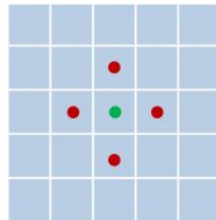
L2: $1.073G \times 0,955 \times 15 = 15,4G$ cycles

CPU Datasheet:

- Cache L1D: 48 KB (x56 cores)
- Cache L2: 2 MB (x56 cores)
- Cache L3: 105 MB (2x node)

cycles per cache hit:

- L1: 5 cycles
- L2: 15 cycles
- L3: 100 cycles
- RAM: 350 cycles



Some math

Strong scaling

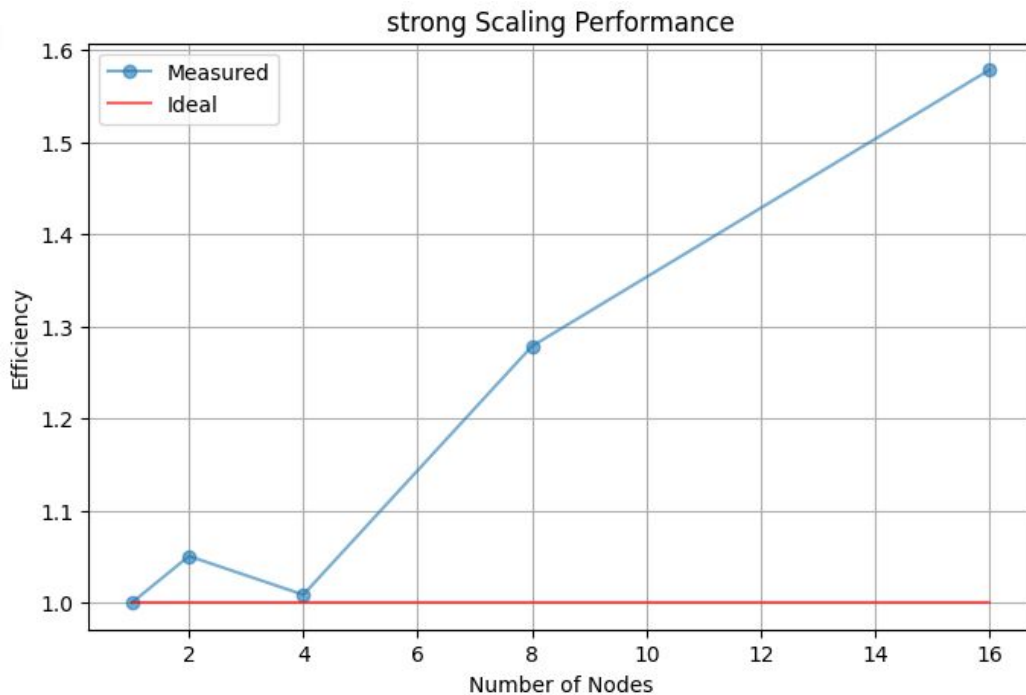
1 NODE:
94,292 % of all cache refs

2 NODES:
89,352 %

4 NODES:
79,325 %

8 NODES:
43,668 %

16 NODES:
4,519 %



Weak scaling

1 NODE:

```
tot time,compute, border, comm  
7.044116,7.039394,1.557733,2.305989
```

2 NODES:

```
7.427694,7.422025,0.756903,2.295715
```

4 NODES:

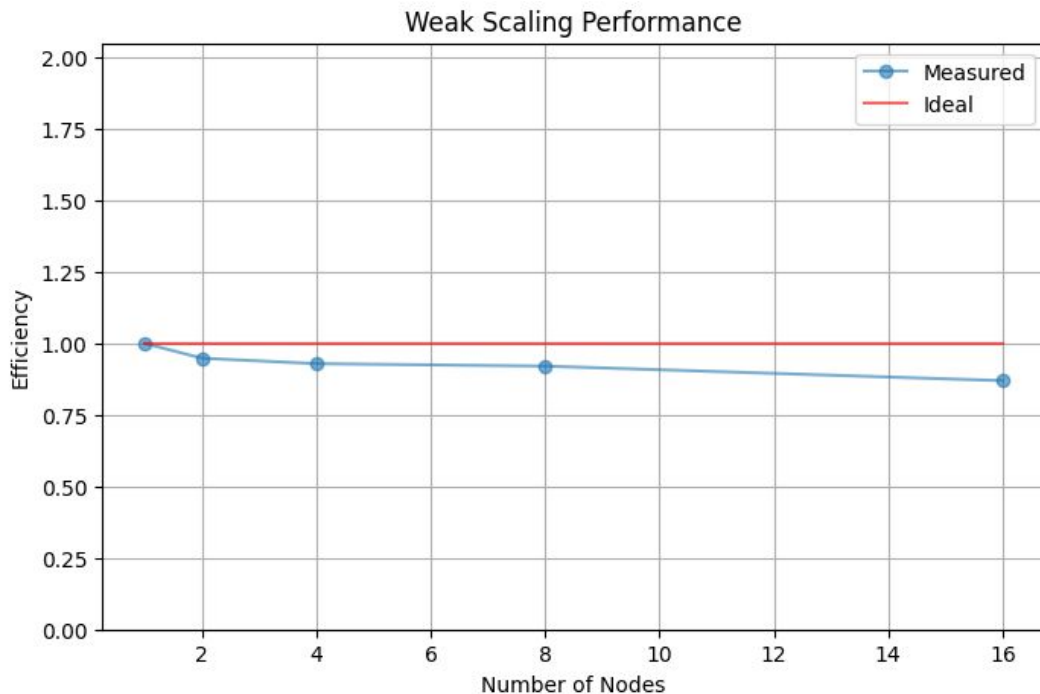
```
7.579342,7.574515,1.579943,3.019374
```

8 NODES:

```
7.649984,7.644082,0.758724,2.868116
```

16 NODES:

```
8.097255,8.089692,1.590328,3.917705
```



What if you saturate L2 cache?

32k-side matrix test



Thread scaling

Run 1

grid 32768*32768

thread:

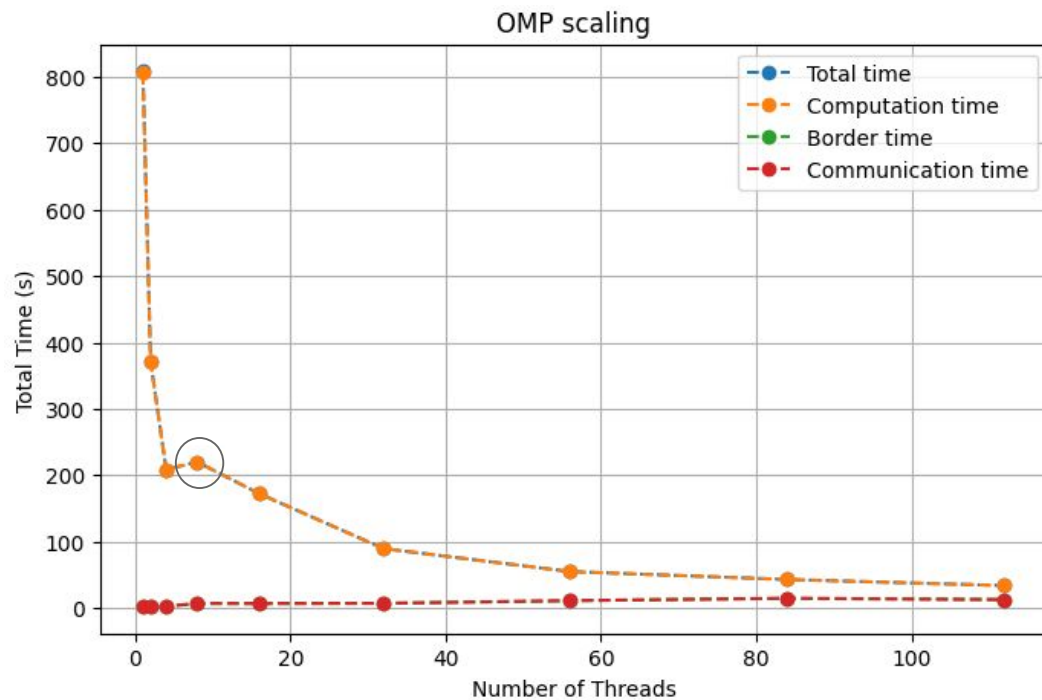
1, 2, 4, 8, 16, 32, 56, 84, 112

NODES=1

N_TASKS_PER_NODE=1

TOTAL_TASKS=1

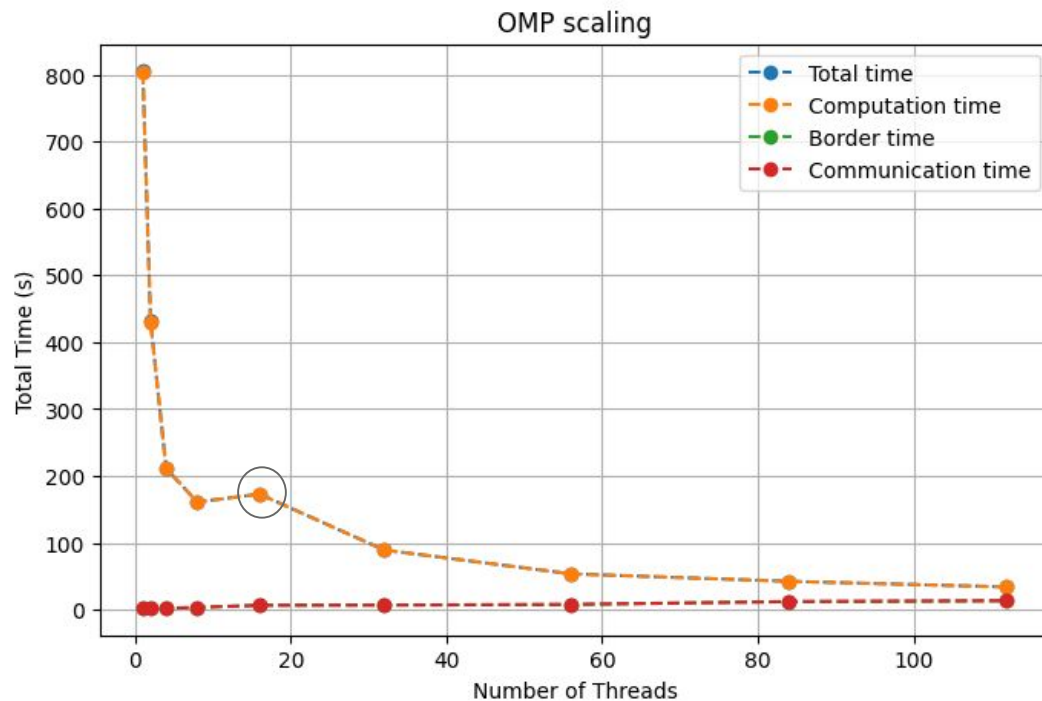
N_STEPS=500



Thread scaling

Run 2

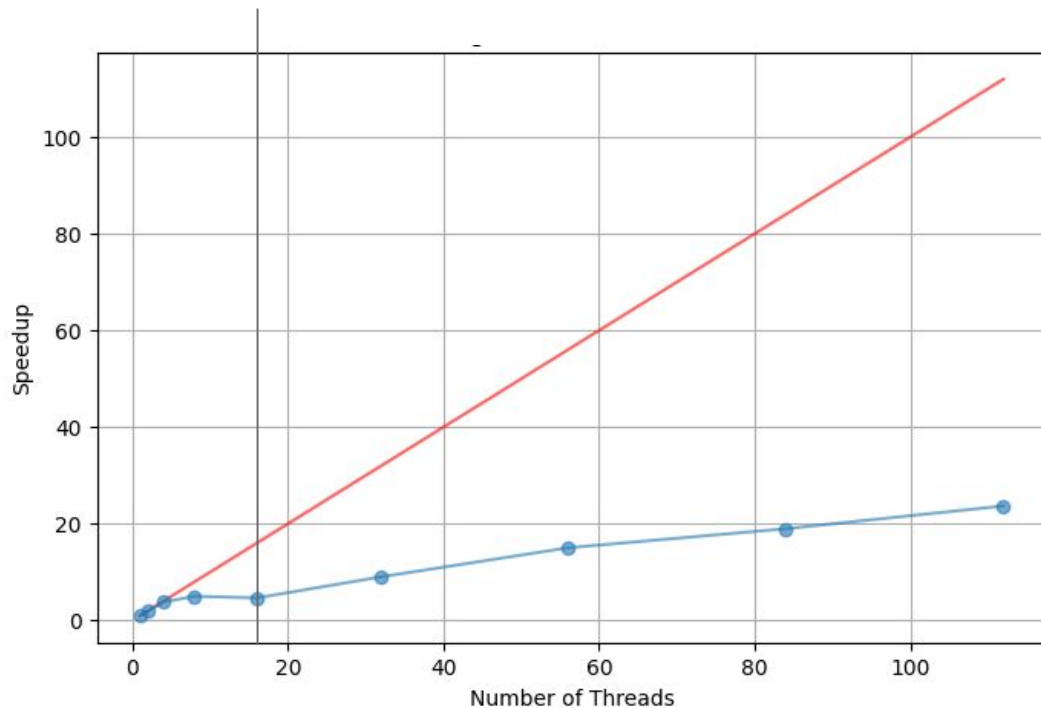
```
NODES  MODELIST( REASON)
16  (Priority)
8  (Priority)
2  (Priority)
4  lrdn[3575-3577,3587]
1  lrdn4884
```



Thread scaling

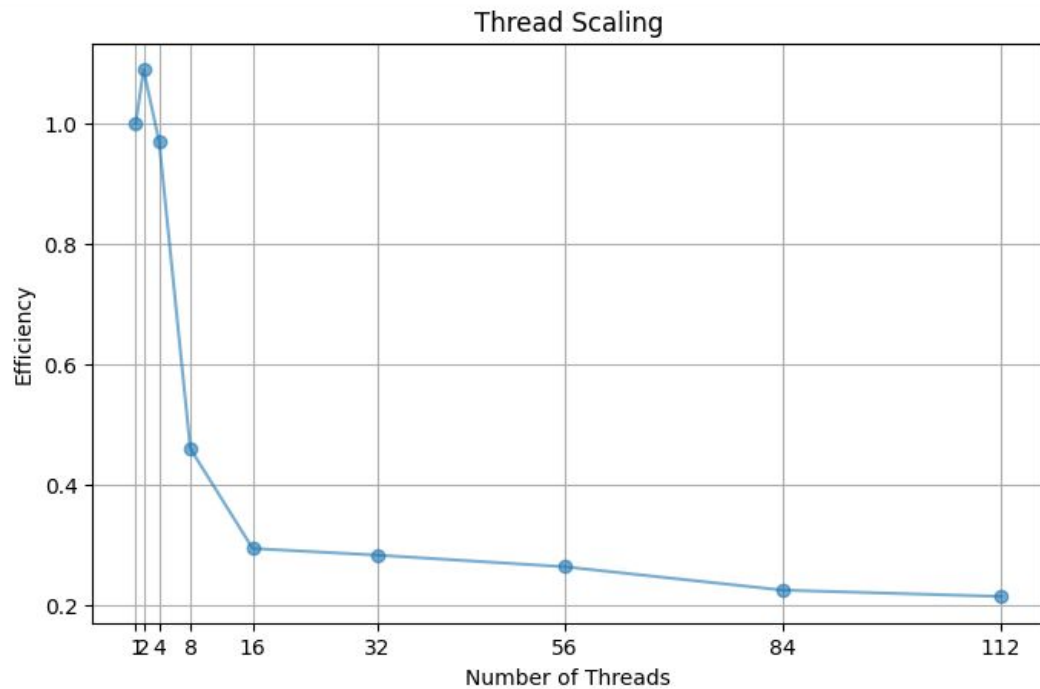
Run 1

- Saturation of busses to ram;
- same small increase after 14 due to more numa regions being used.



Thread scaling

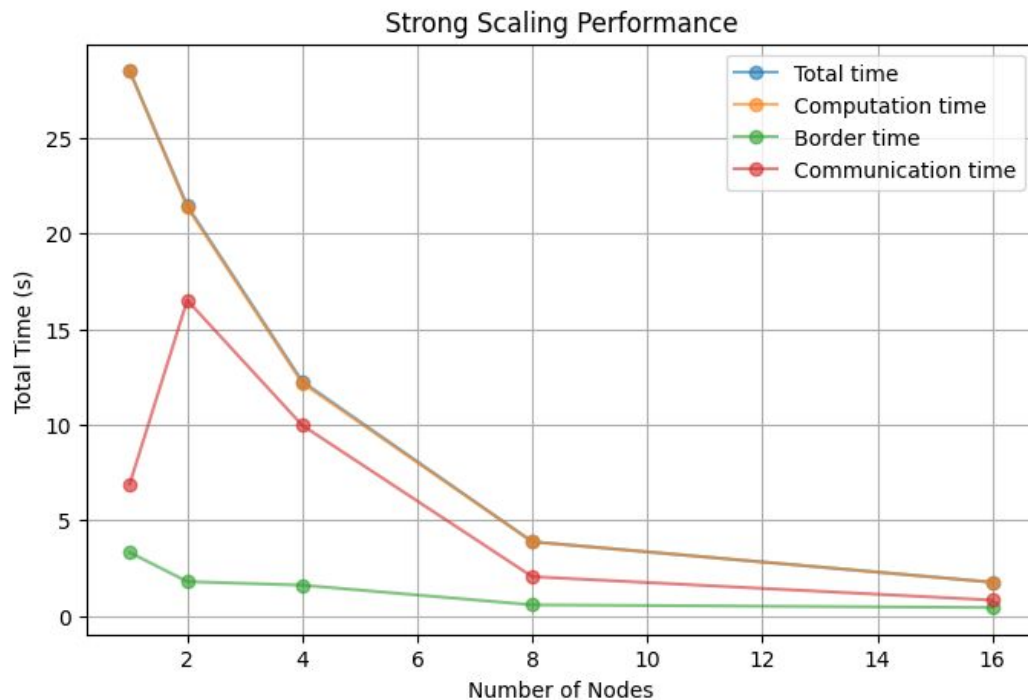
Run 1



Strong scaling

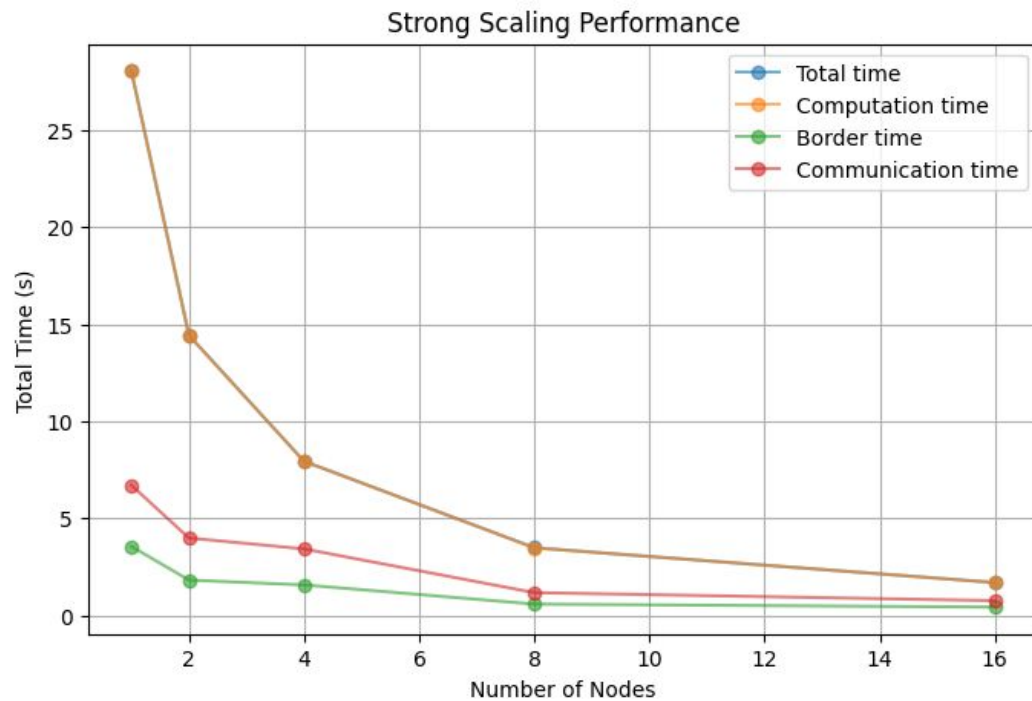
Run 1

N_STEPS=500
GRID_SIZE_X=32768
GRID_SIZE_Y=32768
OMP_THREADS=14
CORES_PER_NODE=112
NTASKS_PER_NODE=8



Strong scaling

Run 2



Strong scaling

Run 1

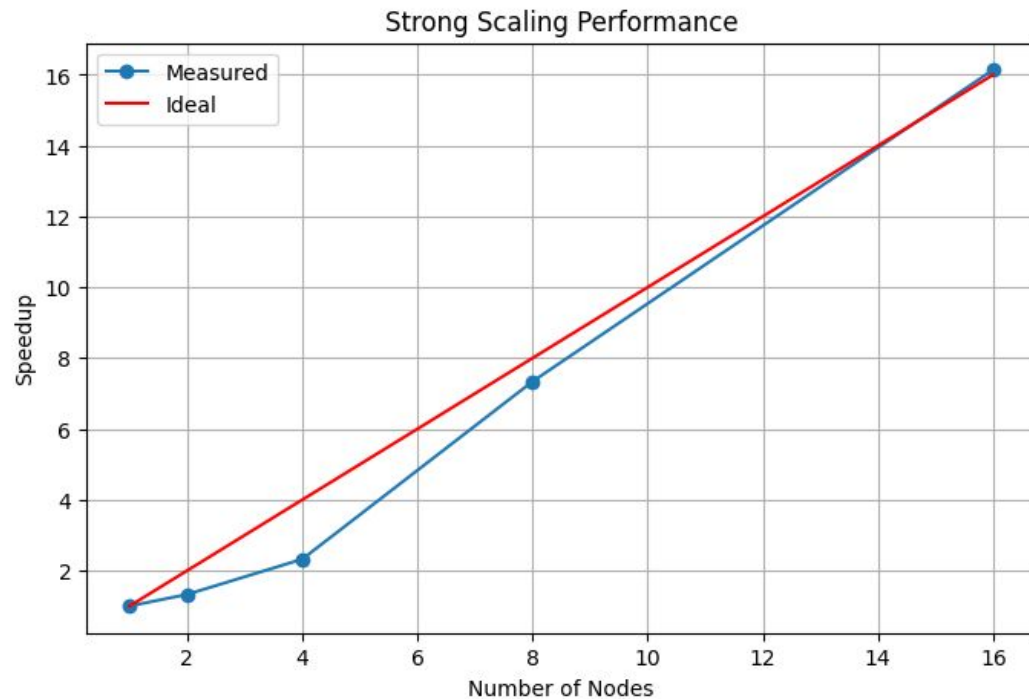
1 NODE:
97,836 % of all cache refs

2 NODES:
96,810 %

4 NODES:
94,049 %

8 NODES:
89,356 %

16 NODES:
79,069 %



Strong scaling

Run 2

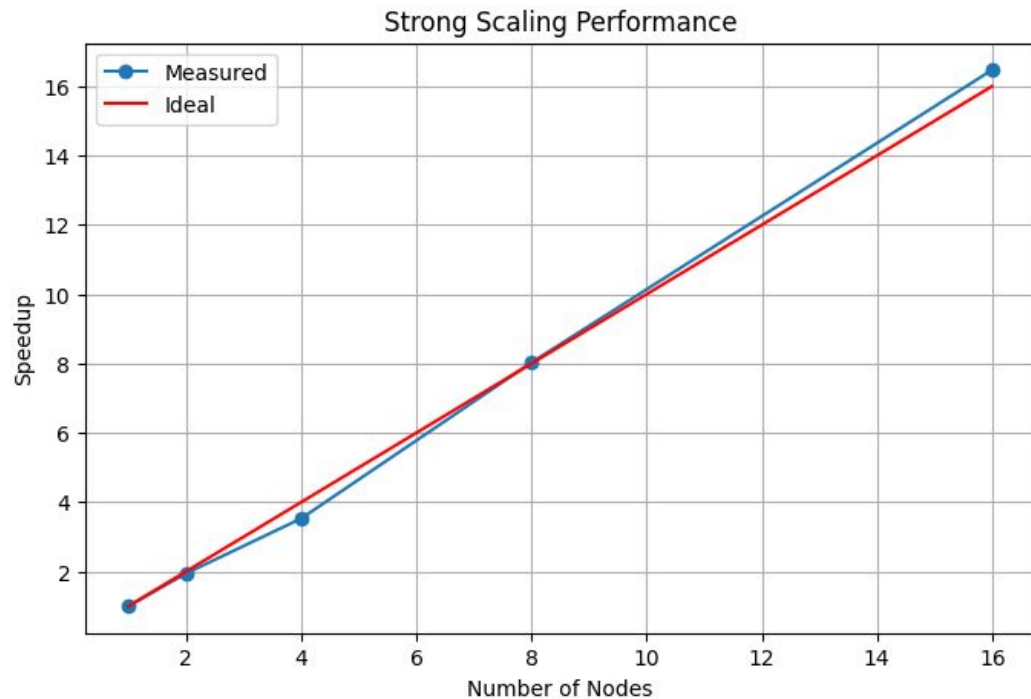
1 NODE:
97,539 % of all cache refs

2 NODES:
96,651 %

4 NODES:
94,217 %

8 NODES:
89,316 %

16 NODES:
79,213 %



weak scaling

Run 2

1 NODE:

tot time,comput, border, comm

28.173501,28.166898,3.449287,6.888591

2 NODES:

29.202034,29.191879,2.076759,7.788567

4 NODES:

30.787232,30.763590,3.687981,11.050531

8 NODES:

30.533302,30.516701,1.954246,10.070392

16 NODES:

32.072737,32.033697,3.592540,13.260953

